

KeenPay

Agentic commerce with bounded AI

KeenPay

Project overview

01 September 2026

<https://github.com/ambitiouss22/KeenPay>

KeenPay

Agentic checkout with a hard boundary between what the AI can suggest and what the system is allowed to charge.

Stack: FastAPI, LangGraph, PostgreSQL, Redis, Next.js, Razorpay (test mode for dev).

What it does

Three jobs, one pipeline:

- Grow — cross-sell, upsell, bundle suggestions from catalog + purchase context
- Sell — buyer finds a product, negotiates, confirms, pays via Razorpay Payment Link
- Protect — every discount, hold, and payment call passes deterministic Python rules first

The rule we do not break:

The LLM proposes. The policy engine approves. Only then does Razorpay get called.

No API keys in the model context. No `create_payment_link` tool on the agent. Payment is a gated side effect, not a model decision.

Repo layout

```
KeenPay/
├── api/                # FastAPI + LangGraph backend
├── frontend/           # Next.js – chat + trace panel
├── workers/            # Webhook processor, hold expiry, reconciliation
├── db/                 # Migrations + dev seeds (DDL canonical: docs/SCHEMA.sql)
├── deploy/compose/     # Docker Compose (local, dev, CI test)
├── tests/e2e/          # Cross-service Playwright flows
├── docs/               # PRD, architecture, API spec, guardrails
├── scripts/            # Bootstrap, PDF generators
├── STRUCTURE.md        # Full directory map + layer boundaries
├── Makefile            # dev, test, bootstrap shortcuts
└── .env.example        # Required environment variables
```

See `STRUCTURE.md` for the complete production directory map, module boundaries, and what is intentionally excluded (no AWS/cloud IaC for now).

Checkout flow (short)

```
chat message
-> parse_intent (LLM)
-> catalog_search (Postgres full-text, not LLM)
-> negotiate_offer (LLM proposes discount % only)
-> guardrail_check (PolicyEngine – sync Python, no LLM)
-> compute_totals (integer parse math)
-> await_user_confirmation (LangGraph interrupt)
-> create_payment_link (Razorpay, idempotent)
-> webhook marks order paid
```

Trace events stream over WebSocket to the right-hand panel while `audit_logs` stores the durable record.

Non-negotiables

These are enforced in code, not in prompts:

Rule	How
Max discount	<code>RULE_MAX_DISCOUNT</code> clamps or rejects
Margin floor	<code>RULE_MIN_MARGIN</code> uses <code>products.cost_paise</code>
Stock	<code>RULE_INVENTORY_AVAILABLE</code> + <code>inventory_holds</code>
No blind retry on timeout	order stays pending/unknown; reconcile against Razorpay
Idempotency	<code>(session_id, offer_version)</code> on payment links; <code>event_id</code> on webhooks
Audit	append-only <code>audit_logs</code> with DB trigger blocking UPDATE/DELETE

Full rule list: `docs/GUARDRAILS_AND_SAFETY.md`.

Local setup

```
# 1. Environment
cp .env.example .env          # fill Razorpay test keys + OPENAI_API_KEY

# 2. Database + seed (Windows)
.\scripts\dev\bootstrap.ps1

# 3. Run services
make dev-api                  # API on :8000
make dev-web                  # Frontend on :3000

# 4. Full checkout test (API)
cd api && pip install -e ".[dev]"
pytest tests/integration/test_checkout_flow.py -v
```

Shell

Dev login: `shopper@keenpay.dev` / `KeenPayDev1!`

Open `http://localhost:3000` → Start checkout session → chat + trace UI.

For tests without Postgres: `USE_IN_MEMORY_STORE=true pytest`

Docs worth reading first

- 1 `docs/PRD.md` — what ships in v1
- 2 `docs/ARCHITECTURE.md` — graph nodes and state shape
- 3 `STRUCTURE.md` — where code lives and layer boundaries
- 4 `docs/AI_JUDGMENT.md` — if you are wiring the agent, read this before adding tools